



Programming b

SPACE MATH



Abstract

I dette projekt har vi udviklet et matematikspil kaldet *SpaceMath*, som er lavet til elever i de mindre klasser. Formålet med spillet er at gøre det sjovere at lære matematik, især addition og subtraktion, gennem et interaktivt spil.

Spillet er lavet i Python med Pygame og er bygget op efter trelagsmodellen, hvor vi har delt programmet op i et data-, logik- og præsentationslag. Det har gjort det nemmere for os at holde styr på koden og få de forskellige dele til at arbejde sammen.

I spillet får eleverne en række matematikopgaver, som de skal løse. Undervejs får de feedback på deres svar, og til sidst får de en samlet score. Når spillet er færdigt, bliver elevens resultat gemt i en SQLite-database, så man kan gemme data over tid.

En vigtig del af vores projekt har været arbejdet med data. Vi har fokuseret på, hvordan vi kan indsamle data fra eleverne, gemme det og bruge det bagefter. Vi har blandt andet lavet en funktion, hvor læreren kan se en oversigt over elevernes resultater, som fx hvor mange spil de har spillet, og deres samlede score. På den måde kan læreren følge med i elevernes udvikling.

Vi er ikke nået helt i mål med alle vores idéer, fx multiplayer-delen, men vi har stadig fået lavet et fungerende spil med de vigtigste funktioner. Alt i alt viser projektet, hvordan man kan kombinere programmering, data og læring i et spil

Indhold

Opgaveformulering Tema 1	4
Gruppefordeling	7
Fokus på data i vores projekt	8
Hvordan vi valgte database	8
Hvordan vi indsamler data fra elever	8
Hvordan data bruges i koden	9
Hvordan data bruges af læreren	10
Hvordan data flyder gennem hele systemet	11
Brug af AI	12
Test	13
Konklusion	13

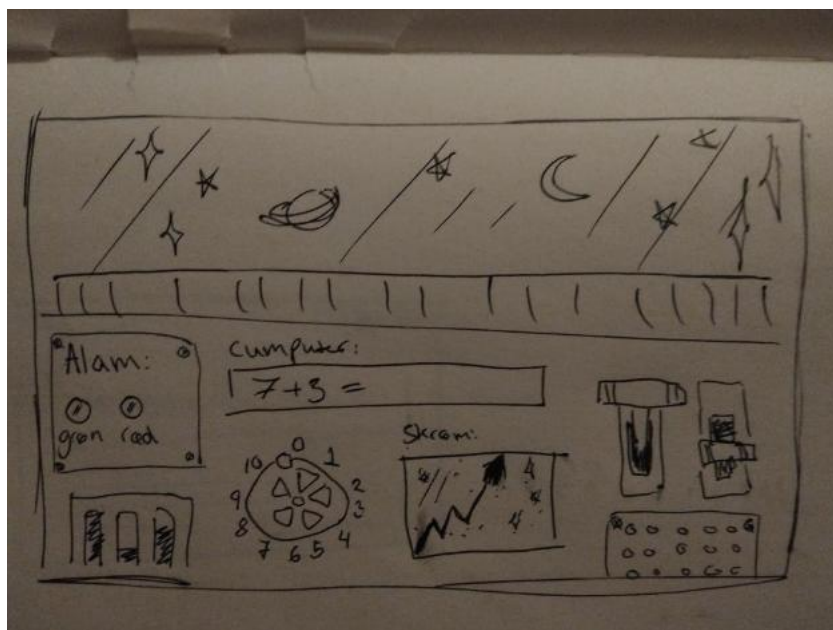
Opgaveformulering Tema 1

SpaceMath, Multispiller matematik spil med læring

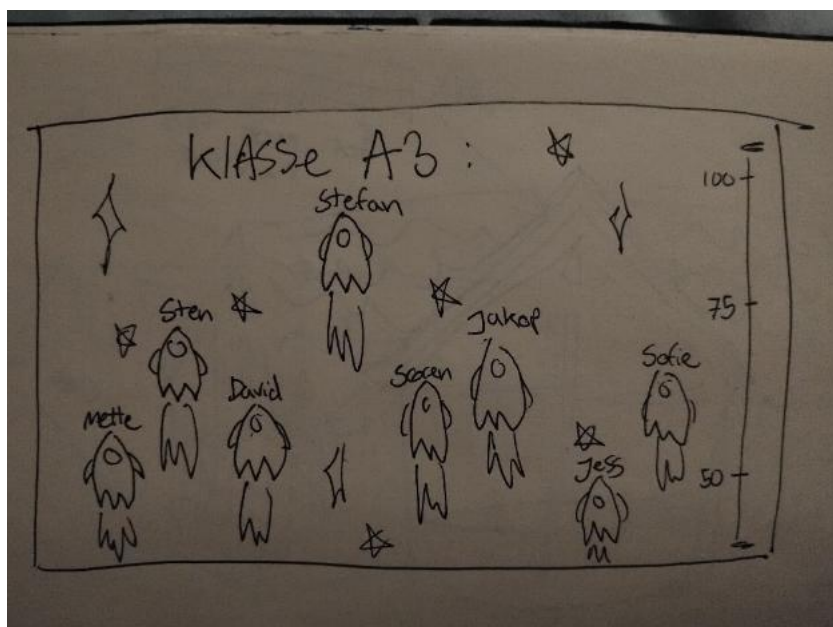
Vi ville lave et interaktiv hjemmeside/spil til læring som er målrettet for elever i alderen 5–9 år. Formålet er at styrke elevernes hjerne med matematiske regne stykker gennem et motiverende miljø hvor de kæmper mellem hinanden.

Selve spillet er designet som et 2D rum eventyr hvor eleverne styrer en rumraket ved at løse matematikopgaver. Spillet er så et rumtema, hvor eleven er inde i rumrakets kontrolpanel. For så at få raketten til at flyve frem skal eleven løse regnestykker korrekt for at få nok brændstof. De matematik opgaverne vise med forskellige signaler som grøn lys som betyder at der skal tilføjes brændstof (plus regne stykker) og et rødt lys som betyder, at der skal fjernes brændstof (minus regne stykker). Eleven afgiver så deres svar ved at justere et kontrolhjul frem og tilbage. Det er et multispiller spil hvor flere elever kan deltage i samme spil samtidig. En lærer opretter en spil session og genererer en kode, som eleverne skal bruge til at spille med i spillet fra deres egne enheder. Eleverne konkurrerer så mod hinanden ved at løse opgaver hurtigst og mest korrekte. Læreren der opretter spil kan vælge sværhedsgrad, overvåge elevernes progression og se resultater og statistiker over spillet. Derudover har læreren en funktion hvor de kan del deres skærm med stor skærmen eller andet i undervisningen for at se alle elevers raketter flyve side om side med hinanden

.
Dette er et spil skoler kan købe for at få adgang til det, ved abonnement får hver lærer oprettet en special bruger som de så kan bruge til undervisning samt alle deres elever få en kode hver de skal logge ind med hvor så deres proces/ resultater kommer til at stå.



Figur 1 Elevernes skræm under spillet



Figur 2 Lærernes skræm under spillet på evt. en stor skræm

Projektets milepæle

Milepæl 1: One-player game med database (Vigtigt)

En simpel version af spillet hvor kun en spiller kan spille, med matematikopgaver og lagring af data.

Krav:

- En person skal kunne spille spillet alene.
- Systemet skal generere matematikopgaver med forskellige sværhedsgrader.
- Eleven skal kunne afgive svar gennem en interaktiv kontrol.
- Systemet skal give feedback på rigtige og forkerte svar.
- Elevens score og progression skal gemmes i et database.
- Spillet skal udvikles i pygame med simpel grafik og styring.

Milepæl 2: Oversigtsskærm til lærer (vigtigt)

En del af spillet der er egnede kun til læreren, hvor læreren kan se data fra spillet.

Krav:

- Læreren skal kunne se elevernes resultater og progression.
- Systemet skal præsentere data struktureret og visuelt.
- Læreren skal kunne se en samlet oversigt over spillerens status.

Milepæl 3: Multiplayer funktion

Flere elever kan spille samtidig i samme session.

Krav:

- Læreren skal kunne oprette en spilsession.
- Systemet skal generere en adgangskode til spillet.
- Elever skal kunne spille spillet via koden.
- Elever skal kunne konkurrere mod hinanden i realtid.
- Læreren skal kunne vise alle spilleres raketter samlet på en skærm.

Milepæl 4 Login system

Brugerhåndtering for at opdele lærere og elever.

Krav:

- Brugeren skal kunne logge ind som elev eller lærer.
- Læreren skal have en personlig brugerprofil.
- Elevers data og progression skal gemmes på deres bruger.
- Systemet skal håndtere dataforbindelse mellem database og spil.

Gruppefordeling

Jessica stod for dataen, hvordan vi fandt databasen, hvordan vi anvendte dataen i kode.

Marie stod for de to andre lag som var presentation og logik, samt størstedelen af kodningen.

På grund af sygdom og udfordringer i gruppen kom Marie til at stå for kodningen. Jeg har haft fokus på databasen og data-delen, som også er det, jeg har valgt at arbejde videre med i denne synopsis.

Fokus på data i vores projekt

Noget af det vi har arbejdet meget med, er data. Altså hvordan vi gemmer elevernes resultater, og hvordan vi kan bruge dem bagefter. Derfor er mit fokus punkt i denne synopse data.

Hvordan vi valgte database

Vi har brugt en SQLite-database til at gemme data.

Vi valgte det, fordi det virker ret simpelt i Python, og vi fandt nogle eksempler på det online. Vi overvejede ikke så mange andre løsninger, fordi det her virkede til at være det nemmeste at komme i gang med.

I databasen gemmer vi ting som:

- elevens navn
- score
- hvor mange spil de har spillet

Hvordan vi indsamler data fra elever

Data er en vigtig del af vores projekt, og det er noget af det, vi har haft meget fokus på. Vi har prøvet at forstå, hvordan man kan indsamle og gemme data fra brugeren uden at gøre det besværligt for dem.

Data bliver samlet automatisk, når eleven spiller. Det vil sige, at eleven ikke selv skal gøre noget for at gemme data, det sker i baggrunden.

Hver gang eleven svarer på en opgave, bliver det registreret i programmet. Programmet gemmer ikke nødvendigvis hvert enkelt klik med det samme, men det bruger informationen til at holde styr på elevens score og hvor mange opgaver der er løst. Når spillet er færdigt, bliver de vigtigste data samlet og gemt i databasen.

Dataen bliver først ”behandlet” i logiklaget (hvor svar bliver tjekket og score bliver regnet ud), og derefter sendt videre til datalaget, hvor det bliver gemt permanent. På den måde arbejder de forskellige lag sammen.

Det hele sker “bag kulisserne”, så brugeren ikke rigtig lægger mærke til det. Men selvom det ikke er noget brugeren kan se, så er det en vigtig del af programmet, fordi det gør det muligt at gemme resultater og bruge dem senere, fx i læreroversigten.

```
46
47 #database bruges til at gemme spillers data
48     self.db = Database()
49     self.db.connect()
50
51 #opretter elve i database og gemmer id
52     self.student_id = self.db.add_student(student_name)
53
54 #hele spilrunde logic
55     self.game_session = GameSession()
56
```

Figur 3 Her oprettes eleven i databasen

I presentation.py oprettes forbindelsen til databasen, og eleven bliver oprettet med et navn, hvilket kan ses i figur 3. På den måde kan programmet koble data og score til den rigtige elev.

```
51
52 #kernen i spillet, tjekker svar, opdaterer scorer, tæller løste opgaver, genererer ny opgave
53 def submit_answer(self, guess: int) -> dict:
54
55     correct = check_answer(self.current_problem, guess)
56     self.score = update_score(self.score, correct)
57     self.problems_solved += 1
58
59     feedback = "Rigtigt!" if correct else "Prøv igen."
60     is_complete = self.problems_solved >= self.PROBLEMS_PER_ROUND
61
62     if not is_complete:
63         self.current_problem = generate_problem()
64
65     return {
66         "correct": correct,
67         "feedback": feedback,
68         "score": self.score,
69         "problems_solved": self.problems_solved,
70         "is_complete": is_complete
71     }
72
```

Figur 4 Her bliver elevens svar behandlet og scoren opdateret

I logik.py bliver elevens svar behandlet i logiklaget, hvilket kan ses i figur 4. Programmet tjekker om svaret er rigtigt, opdaterer scoren og tæller hvor mange opgaver eleven har løst. Det er derfor den del, hvor data bliver behandlet, før det senere bliver gemt. Funktionen returnerer også data tilbage, som bruges til at vise feedback til brugeren.

Hvordan data bruges i koden

Data bliver brugt hele tiden i spillet, selvom det ikke altid er noget man lægger mærke til.

Når en elev spiller:

- 1) spillet laver en opgave
- 2) eleven svarer
- 3) programmet tjekker om det er rigtigt
- 4) så bliver scoren opdateret

5) og til sidst bliver det gemt i databasen

På den måde bestemmer logikken hvad der skal ske, og databasen gemmer det bagefter.

```
def save_game(self, student_id: int, score: int, problems_solved: int) -> None:
    assert self._conn is not None
    cur = self._conn.cursor()
    cur.execute(
        "INSERT INTO games (student_id, score, problems_solved) VALUES (?, ?, ?)",
        (student_id, score, problems_solved),
    )
    self._conn.commit()
```

Figur 5 Her gemmes elevens resultat i databasen

I data.py gemmer vi elevens score i databasen, hvilket kan ses i figur 5. Funktionen bruger en SQL kommando (INSERT INTO) til at sætte data ind i tabellen "games". Det er fx elevens ID, score og hvor mange opgaver de har løst.

Databasen fungerer lidt som en tabel med rækker, hvor hver række er et spil. Det her hører til datalaget, fordi det er her vi gemmer data, så det ikke forsvinder når programmet lukkes. Vi bruger til sidst commit(), som sørger for at dataen faktisk bliver gemt.

INSERT INTO games betyder her at data bliver gemt i databasen

score = elevens point

student_id = hvem det er (i vores tilfælde hedder vores student_id "Elev")

problems_solved = hvor mange opgaver

Hvordan data bruges af læreren

Dataen er ikke kun noget, der bliver gemt, det bliver også brugt bagefter, især af læreren. Det har været en vigtig del af vores ide, at spillet ikke kun er for eleven, men også kan give læreren et overblik.

Når eleverne spiller, bliver deres resultater gemt i databasen. Det betyder, at læreren senere kan hente data frem og se, hvordan eleverne har klaret sig.

I vores program har vi lavet en lærerfunktion, hvor man kan se en oversigt over alle elever. Her kan læreren fx se:

- hvor mange spil en elev har spillet
- elevens samlede score

Det fungerer ved, at programmet henter data fra databasen med en SQL-forespørgsel og viser det i brugerfladen. Det betyder altså, at datalaget står for at hente dataen, mens præsentationslaget viser det til læreren.

Det gør, at læreren kan bruge dataen til at få et bedre overblik over elevernes niveau og udvikling. Selvom det er en ret simpel løsning, viser det hvordan data kan bruges i undervisning.

Vores fokus har været at få dataen til at gå hele vejen fra eleven spiller, til at læreren kan se resultatet. Så selvom vi ikke har lavet alle funktioner færdige, har vi stadig fået lavet en form for data-flow, hvor data bliver indsamlet, gemt og brugt igen.

```
99
100 #jeg bruger JOIN til at kombinere data fra flere tabeller til læreroversigten
101 def get_all_students(self) -> list[sqlite3.Row]:
102     assert self._conn is not None
103     cur = self._conn.cursor()
104     cur.execute(
105         """
106         SELECT
107             s.id,
108             s.name,
109             COUNT(g.id) AS games_played,
110             COALESCE(SUM(g.score), 0) AS total_score
111         FROM students s
112         LEFT JOIN games g ON s.id = g.student_id
113         GROUP BY s.id, s.name
114         ORDER BY s.name
115         """,
116     )
117     return cur.fetchall()
118
```

Figur 6 Her hentes data til læreroversigten fra databasen

I data.py henter programmet data fra databasen til læreroversigten, hvilket kan ses i figur 6. Funktionen bruger en SQL-forespørgsel (SELECT) til at hente information om eleverne.

Der bliver også brugt JOIN, som gør at data fra to tabeller (students og games) bliver sat sammen. På den måde kan programmet både vise elevens navn og deres resultater i samme oversigt.

Det gør altså at dataen bliver samlet og gjort klar, før det bliver vist til læreren. Det er derfor en del af datalaget.

Hvordan data flyder gennem hele systemet

Noget af det vi har arbejdet med i projektet, er hvordan data bevæger sig gennem hele programmet. Altså fra eleven spiller, til det bliver gemt og senere vist igen.

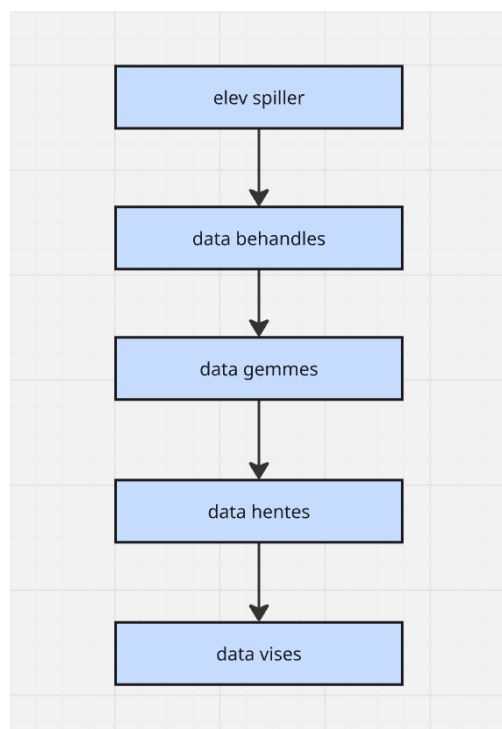
Når eleven spiller spillet, starter det i præsentationslaget, hvor eleven ser opgaven og giver et svar. Det input bliver så sendt videre til logiklaget.

I logiklaget bliver svaret behandlet. Her bliver det tjekket, om det er rigtigt eller forkert, og scoren bliver opdateret. Jeg forstår det som, at det er her programmet “tænker” og beslutter, hvad der skal ske.

Når spillet er færdigt, bliver de vigtigste data (som score og antal opgaver) sendt videre til datalaget. Her bliver det gemt i databasen, så det ikke går tabt.

Senere kan dataen blive hentet igen fra databasen, fx når læreren åbner oversigten. Her bliver data sendt fra datalaget tilbage til præsentationslaget, så det kan vises på skærmen.

Der er derfor et flow i vores spil som ses i figur 7



Figur 7 Flowchart lavet i miro

Brug af AI

I vores projekt har vi brugt AI som hjælp undervejs. Vi har blandt andet brugt ChatGPT og GitHub Copilot til at få forklaret ting, vi ikke helt forstod, og til at finde løsninger på problemer i koden.

Vi har især brugt AI til at forstå, hvordan databaser fungerer, og hvordan vi kunne gemme og hente data med SQL. Det har også hjulpet os med at finde fejl og få idéer til, hvordan vi kunne strukturere vores kode.

Vi har samtidig også fået hjælp af AI til at lave noget af vores kode. Vi har inden eksameprojektet lært hvordan man bruge AI fornuftigt når man koder. Vi har fået inspiration til kode fra AI, men vi har hele tiden prøvet at forstå det og tilpasse det til vores eget program.

Test

Vi har testet programmet ved selv at spille spillet og prøve forskellige situationer.

Vi har testet om:

- 1) elevens score bliver opdateret korrekt
- 2) data bliver gemt i databasen efter spillet
- 3) data kan hentes igen og vises i læreroversigten

Vi har også prøvet både rigtige og forkerte svar for at se, om programmet reagerer korrekt. Hvilket det gjorde.

Det meste fungerer som det skal, men der kan stadig være små fejl, som vi ikke har opdaget.

Konklusion

I dette projekt har vi udviklet et matematikspil med fokus på data. Vi har arbejdet med, hvordan data bliver indsamlet fra eleverne, behandlet i programmet og gemt i en database, så det senere kan bruges igen.

Vi har fået en bedre forståelse for trelagsmodellen og hvordan data, logik og præsentation hænger sammen i et program. Især arbejdet med databasen og data-flowet har været en vigtig del af vores læring.

Selvom vi ikke nåede alle vores oprindelige mål, som fx multiplayer, har vi stadig fået lavet et fungerende spil med de vigtigste funktioner.

Projektet har også givet mig en bedre forståelse for, hvordan data kan bruges i praksis i programmering.

Alt i alt viser projektet, hvordan man kan bruge data i et program til både at gemme resultater og give læreren et overblik over elevernes udvikling.

Bilag

Kode: <https://github.com/neg9414/Eksame-projekt--SpaceMath>

Figur liste

Figur 1 Elevernes skrøm undrer spillet	5
Figur 2 Lærernes skrøm undrer spillet på evt. en stor skrøm	5
Figur 3 Her oprettes eleven i databasen	9
Figur 4 Her bliver elevens svar behandlet og scoren opdateret.....	9
Figur 5 Her gemmes elevens resultat i databasen	10
Figur 6 Her hentes data til læreroversigten fra databasen	11
Figur 7 Flowchart lavet i miro	12